FestaGestor

PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

Enfoque DevOps — Integração e Entrega Contínuas (CI/CD)

Versão 1.0

Campo	Descrição
Projeto	FestaGestor (Sistema ERP para gestão de aluguel de artigos para festas)
Gerente de Projeto	Gustavo Elias
Gerente/Responsável de GCS-DevOps	Gustavo Elias
Fábrica de Software / Squad	Desenvolvedor Individual (Projeto Pessoal)
Repositório principal	https://github.com/gustavoelias11/festagestor

Histórico de Revisões

Data	Versão	Descrição	Autor
19/09/2026	1.0	Elaboração do documento inicial de GCS com definição de branching, commits e planejamento de CI/CD (GitHub Actions e Docker).	Gustavo Elias

Sumário

2
1. Introdução **3**
1.1 Finalidade 3
1.2 Escopo 3
1.3 Definições, Acrônimos e Abreviações 3
1.4 Referências 3
1.5 Visão Geral 3
2. Cultura DevOps e Papéis na Gerência de Configuração **4**
3. Ferramentas, Ambientes e Infraestrutura **4**
3.1 Ferramentas do Pipeline 4
3.2 Ambientes e Infraestrutura 4
3.3 Estratégia de Branching e Versionamento 4
4. Identificação da Configuração **5**
4.1 Itens de Configuração 5
4.2 Convenção de Nomenclatura 5
4.3 Estrutura do Repositório 5
5. Integração Contínua (CI) **6**
5.1 Etapas do Pipeline de CI 6
5.2 Política de Pull Request e Revisão de Código 6
5.3 Critérios de Qualidade (Quality Gate) 6
6. Entrega e Implantação Contínua (CD) **6**
6.1 Etapas do Pipeline de CD 6
7. Infraestrutura como Código (IaC) **6**
8. Controle de Configuração e Mudança **7**
8.1 Processo de Solicitação de Mudança 7
8.2 Comitê de Controle de Mudança (CCM) 7
9. Baselines e Releases **7**
9.1 Geração Automática de Baseline 7
9.2 Notas de Versão (Release Notes) 7
10. Estimativa do Status de Configuração **7**
10.1 Monitoramento e Observabilidade 7
10.2 Métricas DORA 7
10.3 Relatórios e Auditorias 7
11. Marcos **8**
12. Treinamento e Recursos **8**
13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros **8**
14. Quadro-Resumo do Pipeline CI/CD **8**

1. Introdução

Este Plano de Gerenciamento de Configuração (PGC) descreve as atividades de Gestão de Configuração de Software (GCS) que serão executadas no ciclo de vida do **FestaGestor**. O enfoque recai sobre práticas de Integração Contínua (CI) e Implantação Contínua (CD) utilizando repositório único (monorepo), conteinerização e automação de pipelines para reduzir o esforço manual e garantir rastreabilidade.

1.1 Finalidade

Estabelecer um padrão de versionamento, estruturação de branches e controle de mudanças para o desenvolvimento do sistema, preparando o terreno para a implementação futura de pipelines automatizados de build e implantação baseados em Docker e GitHub Actions.

1.2 Escopo

Este PGC destina-se ao desenvolvimento integral do sistema FestaGestor, abrangendo a API backend (Java Spring Boot), o frontend (React/Vite) e o gerenciamento do banco de dados (MySQL via Flyway).

1.3 Definições, Acrônimos e Abreviações

Termo	Significado
GCS	Gestão de Configuração de Software
CI	Integração Contínua (Continuous Integration)
CD	Entrega/Implantação Contínua (Continuous Delivery / Deployment)
IaC	Infraestrutura como Código (Infrastructure as Code)
CCM	Comitê de Controle de Mudanças
Baseline	Conjunto de itens de configuração que atingiram um estado comprovado de estabilidade
Pipeline	Sequência automatizada de etapas de build, teste, análise e implantação
Trunk-based Development	Estratégia de branching em que os desenvolvedores integram na branch principal com alta frequência
SemVer	Versionamento Semântico (MAJOR.MINOR.PATCH) - 1.0.0.
DORA Metrics	Métricas de desempenho DevOps (Lead Time, Deployment Frequency, MTTR, Change Failure Rate)

1.4 Referências

- Documento base: Processos de Desenvolvimento de Software e a Integração com GCS.
- Template do Plano de Gerenciamento de Configuração (RUP 7.0, IBM) — adaptado.
- Repositório do Projeto: https://github.com/gustavoelias11/festagestor.

1.5 Visão Geral

Neste projeto, a GCS foca em manter um repositório monorepo bem estruturado. Como as automações de CI/CD (GitHub Actions) e testes ainda não foram implementadas, o controle inicial se dá por padronização estrita de commits e branches estruturadas (via Pull Requests), evoluindo futuramente para validações automatizadas de pipeline.

2. Cultura DevOps e Papéis na Gerência de Configuração

Como trata-se de um projeto individual, as responsabilidades de DevOps, Desenvolvimento, QA e Operações são centralizadas, com foco em disciplina de versionamento e preparação do ambiente de laboratório para avaliação.

Papel	Responsabilidades
Desenvolvedor(a) / Engenheiro DevOps	Desenvolve o código (Java e React), escreve e gerencia os scripts de migração (Flyway), gerencia as branches, submete e aprova Pull Requests e projetará os futuros pipelines (GitHub Actions) e Dockerfiles.

3. Ferramentas, Ambientes e Infraestrutura

3.1 Ferramentas do Pipeline

Ferramentas sugeridas — substituir pelas efetivamente adotadas pelo projeto:

Categoria	Ferramenta(s) sugerida(s)	Finalidade
Controle de versão	Git + GitHub	Versionamento de código-fonte (monorepo para API e Web).
Orquestração de CI/CD	GitHub Actions *(Planejado)*	Automação de build, testes, análise e implantação
Empacotamento	Docker *(Planejado)*	Geração de imagens/artefatos versionados e imutáveis
Migração de Banco de Dados	Flyway	Controle de versão do esquema do banco de dados (MySQL).

3.2 Ambientes e Infraestrutura

Os ambientes devem ser, sempre que possível, provisionados via Infraestrutura como Código, garantindo paridade entre eles e permitindo recriação rápida e rastreável.

Ambiente	Finalidade	Gatilho de promoção
Desenvolvimento Local	Codificação e testes manuais da API e Interface.	Execução local via IDE / Maven (mvnw) e Vite (npm run dev)
Laboratório / Avaliação	Ambiente de demonstração e verificação final.	Clonagem do repositório, execução dos containers Docker (quando implementados) a partir da branch main.

3.3 Estratégia de Branching e Versionamento

A estratégia adotada é o **GitHub Flow**, focado em Pull Requests iterativos.

- main — Branch principal, contém o código estável e funcional (após merges de PRs).
- feat/<ticket ou nome> — Branches para desenvolvimento de novas funcionalidades (ex: feat/33-tela-gestao-itens).

- Padrão de Commits: Utiliza-se Conventional Commits para clareza no histórico e rastreabilidade, seguindo o padrão já estabelecido no projeto (ex: feat: implementa cadastro..., chore: reestrutura projeto..., fix: corrige validacao...).

Versionamento: adoção de Versionamento Semântico (MAJOR.MINOR.PATCH) e Conventional Commits (feat, fix, chore, docs, refactor…), permitindo geração automática de changelog e da próxima versão pelo pipeline.

4. Identificação da Configuração

4.1 Itens de Configuração

Além do código-fonte, em um contexto DevOps os seguintes artefatos também são itens de configuração formais, versionados no mesmo repositório (ou repositórios compartilhados):

- Código-fonte da aplicação Backend (Java Spring Boot).
- Código-fonte do Frontend (React/Tailwind).
- Scripts de migração de banco de dados (V1__... a V9__... via Flyway em src/main/resources/db/migration).
- Arquivos de propriedades do Spring (application.properties e .example).
- *Futuros:* Definições do GitHub Actions (.github/workflows) e Dockerfile/docker-compose.yml.

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Branch de feature	feat/<id-opcional>-<descrição>	feat/33-tela-gestao-itens
Commit	<tipo>(<escopo opcional>): <descrição>	feat: implementa confirmacao de aluguel

4.3 Estrutura do Repositório

O projeto adota uma estrutura de monorepo, separando claramente as camadas da aplicação em diretórios distintos na raiz do repositório principal:

- **/festagestor-api/**: Diretório exclusivo do Backend (Java Spring Boot), estruturado e gerenciado via Maven (pom.xml, arquivos executáveis mvnw e mvnw.cmd).
  - /src/main/java/...: Código-fonte da API, organizado internamente em pacotes por domínio/feature.
  - /src/main/resources/db/migration: Scripts de configuração e controle de versão do banco de dados geridos pelo Flyway.
- **/festagestor-web/**: Diretório exclusivo do Frontend (React/Vite), gerenciado via NPM contendo as dependências (node_modules, package.json, package-lock.json) e configurações de ambiente (vite.config.js, tailwind.config.js, eslint.config.js, .env).
  - /src/: Código-fonte principal da interface, incluindo App.jsx, componentes, páginas, roteamento e consumo da API.
  - /public/: Arquivos e assets estáticos expostos publicamente.
- **/.github/**: Diretório reservado para configurações nativas do repositório, onde residirão os futuros workflows do GitHub Actions responsáveis pelo pipeline de CI/CD.

- **Arquivos de Raiz**: O diretório base contém a documentação geral do sistema (README.md), além dos arquivos de controle do próprio Git (.git/, .gitignore).

5. Integração Contínua (CI)

Como o pipeline do GitHub Actions ainda não foi implementado e testes automatizados ainda não foram desenvolvidos, o processo de CI ocorre de forma processual através do ciclo de Pull Requests.

5.1 Etapas do Pipeline de CI

- Checkout do código.
- Build do Backend (Maven / mvnw clean package).
- Build do Frontend (Vite / npm run build).
- *(Futuro)* Execução de testes automatizados e análise de código.
- Geração de imagens Docker.

5.2 Política de Pull Request e Revisão de Código

- Todo código deve ser desenvolvido em uma branch secundária (feat/, fix/).
- A integração à branch main só ocorre via aprovação e merge do Pull Request.

5.3 Critérios de Qualidade (Quality Gate)

Até a implementação de testes unitários, o critério de qualidade obrigatório para merge de um PR é:

1. Sucesso na compilação do Backend sem erros.
2. Sucesso no build do Frontend.
3. Migrações do Flyway (V__*.sql) sem conflitos de versão ou erros de sintaxe.

6. Entrega e Implantação Contínua (CD)

Para a entrega atual (Avaliação em Laboratório), a "implantação" consiste em garantir que a branch main possua todas as instruções de execução configuradas.

6.1 Etapas do Pipeline de CD

- A avaliação será executada através de verificação no laboratório.
- O processo de deploy consistirá em clonar a branch main do repositório, configurar o .properties baseando-se no application.properties.example, e rodar o projeto. Quando o Docker for implementado, a execução se resumirá ao comando docker-compose up.

7. Infraestrutura como Código (IaC)

O estado do banco de dados relacional (MySQL) é tratado como código através do Flyway. Nenhuma alteração estrutural na base de dados é feita manualmente; todas devem seguir o fluxo de criação de scripts Vx__descricao.sql no diretório de migrações e serem versionadas junto com a respectiva funcionalidade.

8. Controle de Configuração e Mudança

8.1 Processo de Solicitação de Mudança

Mudanças de rotina (funcionalidades, correções) seguem o fluxo padrão do repositório: criação de uma nova branch (feat/ ou fix/) -> desenvolvimento -> submissão de Pull Request -> aprovação e merge. Esse fluxo, apoiado pelo GitHub, constitui o registro formal e rastreável de qualquer alteração no FestaGestor.

8.2 Comitê de Controle de Mudança (CCM)

Por se tratar de um projeto de escopo acadêmico e desenvolvimento individual, o papel do CCM é exercido pelo próprio desenvolvedor (Gustavo Elias) em alinhamento com os requisitos e avaliações da disciplina. Mudanças de alto impacto (ex.: grandes alterações no modelo de dados via Flyway ou mudanças na arquitetura do monorepo) são validadas exaustivamente em ambiente de desenvolvimento local antes de serem integradas à branch principal.

9. Baselines e Releases

9.1 Geração Automática de Baseline

As baselines do projeto são estabelecidas no fechamento de marcos de entrega. Uma tag de versão (ex.: v1.0.0) será gerada no GitHub apontando para o commit estável da entrega na branch main, "congelando" a versão do código e dos scripts de banco de dados.

9.2 Notas de Versão (Release Notes)

As notas de release serão geradas através da interface do próprio GitHub (GitHub Releases), baseando-se no histórico de *Conventional Commits* integrados desde a última tag gerada.

10. Estimativa do Status de Configuração

10.1 Monitoramento e Observabilidade

Atualmente, o monitoramento ocorre em tempo de desenvolvimento e homologação através dos logs gerados pela aplicação Spring Boot no terminal e pelo console do Vite/React.

10.2 Métricas DORA

Métrica	Descrição
Deployment Frequency	Frequência com que o código é implantado em produção.
Lead Time for Changes	Tempo entre o commit e a implantação em produção.
Change Failure Rate	Percentual de implantações que resultam em falha em produção.
Mean Time to Restore (MTTR)	Tempo médio para restaurar o serviço após uma falha.

10.3 Relatórios e Auditorias

O histórico de pipelines (builds, testes, deploys) e os logs do sistema de versionamento constituem, por si, o principal insumo de auditoria de GCS, dispensando relatórios manuais na maior parte dos casos.

11. Marcos

Os marcos do projeto orientados à GCS e práticas DevOps incluem:

- **Marco 1 (Concluído):** Estruturação inicial do monorepo (diretórios festagestor-api e festagestor-web) e versionamento do banco via Flyway.
- **Marco 2:** Entrega e validação do Plano de Gerenciamento de Configuração e demonstração do sistema em ambiente de laboratório.
- **Marco 3:** Criação e configuração do pipeline de CI utilizando GitHub Actions para automatizar o build do Maven e NPM em cada Pull Request.
- **Marco 4:** Dockerização completa (Backend, Frontend e Banco de Dados) orquestrada via docker-compose.

12. Treinamento e Recursos

A implementação contínua das práticas DevOps neste projeto exige aprofundamento constante. O desenvolvimento da API backend é suportado pelo estudo contínuo do ecossistema Java Spring Boot, abrangendo desde fundamentos como o uso da *Stream API* até conceitos avançados de arquitetura, utilizando plataformas online focadas em programação como DevDojo e Alura. Para as próximas etapas de automação do projeto, o foco prático será direcionado ao estudo da sintaxe YAML para a construção dos fluxos no GitHub Actions e a configuração otimizada de Dockerfiles.

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

As dependências de terceiros no projeto FestaGestor são estritamente controladas pelos gerenciadores de pacotes nativos de cada ecossistema:

- **Backend:** O arquivo pom.xml gerencia e trava as versões das dependências do Maven (ex.: Spring Data JPA, Flyway, JWT).
- **Frontend:** Os arquivos package.json e package-lock.json (ou equivalente gerado no build) garantem as versões exatas das bibliotecas do Node.js/React. Como passo futuro, a ferramenta de SCA (Software Component Analysis) GitHub Dependabot será ativada no repositório para monitorar bibliotecas e sugerir atualizações de segurança automaticamente via Pull Requests.

14. Quadro-Resumo do Pipeline CI/CD

Resumo do fluxo padrão, do commit à produção:

Etapa	Descrição	Gatilho
CI (Planejado)	Build automatizado do Maven (mvnw clean package), build do Vite (npm run build) e futura execução de testes.	Abertura/atualizaç ão de Pull Request

Etapa	Descrição	Gatilho
Merge	Integração da nova funcionalidade estabilizada na branch main.	Aprovação de PR e verificação local sem erros
CD (Avaliação)	Integração da nova funcionalidade estabilizada na branch main.	Aprovação de PR e verificação local sem erros